WalletPK: Distributed Mobile Wallet & Payments Platform

Project Proposal · Nov 8, 2026 · @spideyy

1. Executive Summary

WalletPK is a production-grade mobile wallet and payments platform modelled on JazzCash and Easypaisa. It lets users send money, pay utility bills, top up mobile credit, pay merchants by QR and transfer to any bank over a simulated Raast network.

The project is backend-first. Its core is an immutable double-entry ledger, distributed locking, orchestrated Sagas, the Transactional Outbox pattern and CQRS. Every external party (banks, billers, telcos, NADRA) is a simulated microservice that can be slowed down, broken or killed on demand.

Three frontends make the backend visible: a customer wallet app, an operations and compliance back-office, and a **Simulation Control Center** that generates thousands of real users and injects failures live. The demo goal is simple: show 10,000 concurrent transactions, a crashed bank and a fraud attack, and prove that not one rupee is lost or created.

Core outcomes

- Zero double-spend under concurrent load (verified by a ledger invariant check after every simulation run)
- Zero lost transactions when any downstream service crashes
- Automatic refunds when a biller or bank fails permanently
- Real-time fraud detection and a live operations dashboard
- Entire stack runs on one laptop via docker-compose at $0 cost

2. Problem Statement

Pakistan's mobile wallets process millions of small, high-frequency transactions a day, and their backends fail in ways users feel directly. WalletPK targets five real, recognisable failure modes.

Real-life problem	What the user experiences	WalletPK's engineered answer
Bill payment times out at the biller	"Money deducted but bill not paid"	Payment Saga with retries, then automatic compensating refund
Two transfers fired from the same wallet at once	Negative balance, money created from nothing	Redis distributed lock + optimistic concurrency + DB check constraint
User taps "Send" twice on a slow network	Charged twice	Idempotency-Key with a 24-hour response cache
Bank transfer stuck "in process" for hours	No visibility, support calls	Saga state machine with timeouts, status tracking and push notifications
SIM-swap fraud drains a wallet	Account emptied in minutes	Real-time rules engine on the event stream; auto-freeze + step-up OTP
End-of-day mismatch with banks	Finance team reconciles manually	Automated reconciliation job against simulated bank settlement files

Existing student projects usually build a CRUD wallet that works for one user at a time. WalletPK is designed to stay correct when 5,000 simulated users, a failing bank and a fraud ring all hit it at once.

3. Actors & Personas

Actor	Example persona	What they do in WalletPK
Wallet customer	Ali, a university student in Lahore	Sends money, pays K-Electric/LESCO bills, buys mobile packages, scans QR at shops
Merchant	Bashir's General Store, Karachi	Accepts QR payments, sees daily settlement, withdraws to bank
Retail agent	Easy-cash shop owner in Multan	Cash-in / cash-out for walk-in customers, earns commission

Actor	Example persona	What they do in WalletPK
Operations analyst	WalletPK ops team	Monitors live throughput, stuck Sagas, dead-letter queue, replays failed jobs
Compliance officer	AML/KYC desk	Reviews fraud alerts, freezes accounts, approves KYC upgrades, exports audit trails
Finance team	Treasury	Runs end-of-day reconciliation, views float and settlement positions
Simulator (system actor)	Chaos engineer / examiner	Spawns virtual users, triggers traffic spikes, kills services, launches fraud attacks
External systems (simulated)	Mock Raast, mock banks, billers, telcos, NADRA	Respond with configurable latency, error rates and outages

4. Real-Life Scenarios Simulated

Each scenario is a one-click preset in the Simulation Control Center, with a pass/fail check at the end.

S1: Salary day rush. On the 1st of the month, 5,000 virtual users receive salary credits and immediately send money home, pay bills and top up. Traffic ramps to 500 transactions per second. Pass: p95 latency under target, ledger balanced, zero negative balances.

S2: "Paisay kat gaye, bill pay nahi hua". The K-Electric biller simulator is set to time out 100% of requests. Payments enter the Saga, retry three times with exponential backoff, then compensate. Pass: every affected user is refunded within 3 seconds of final failure and gets a notification.

S3: Double-tap on slow 3G. The simulator replays the same transfer request 5 times with the same Idempotency-Key, under 800 ms network jitter. Pass: exactly one debit, four cached responses.

S4: Race on the last Rs. 1,000. 50 parallel withdrawals hit one wallet holding Rs. 1,000. Pass: exactly the affordable number succeed; balance never goes below zero.

S5: Bank outage during Raast transfer. The mock HBL service is killed mid-transfer. The circuit breaker opens, new transfers queue, and in-flight ones stay in a pending hold. When HBL returns, queued transfers drain automatically. Pass: zero lost or duplicated transfers.

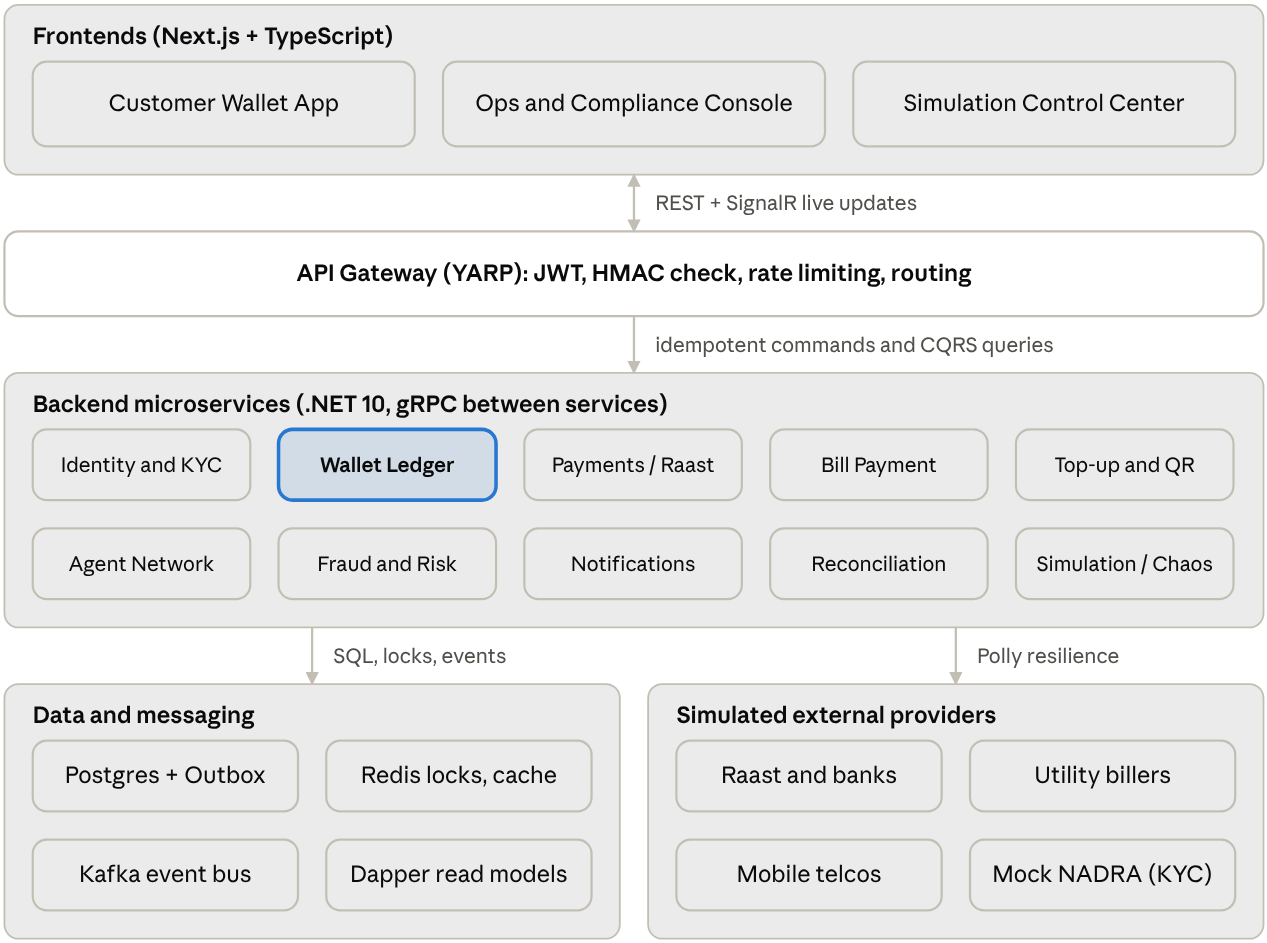
S6: SIM-swap fraud ring. 20 compromised accounts log in from new devices and try to drain funds to 3 mule wallets. Pass: the rules engine flags the pattern, freezes accounts and raises alerts on the compliance dashboard within 2 seconds.

S7: Database or broker crash. Kafka is stopped for 60 seconds while traffic continues. Pass: Outbox keeps all events in PostgreSQL; on recovery every event is delivered, and the dashboards catch up.

S8: End-of-day reconciliation. Mock banks generate settlement files containing deliberate mismatches (missing, duplicate and amount-mismatch entries). Pass: the reconciliation job detects and classifies every injected mismatch.

5. System Architecture

WalletPK is an event-driven microservice system. Commands change the ledger; every change is published as an event; read models, fraud checks, notifications and dashboards all react to those events.



WalletPK architecture · 3 frontends, 10 services, shared data and mock providers

The Wallet Ledger service (highlighted) is the only service allowed to move money. Every other service asks it to place, capture or release holds. The Simulation / Chaos service drives virtual users through the gateway and controls the mock providers' latency, errors and outages.

Request lifecycle (bill payment example)

1. The wallet app sends `POST /bills/pay` with an Idempotency-Key; the gateway validates the JWT and rate limit.
2. The Bill Payment service starts a Saga and asks the Ledger to place a hold on Ali's wallet.
3. The Ledger writes the hold and an `HoldPlaced` outbox event in one PostgreSQL transaction.
4. The outbox relay publishes to Kafka; the Saga calls the mock biller through Polly.

5. On success the Ledger captures the hold (debit wallet, credit biller and fee accounts); on permanent failure it releases the hold.
6. Fraud, Notifications and read-model consumers process the events; SignalR pushes the new status and balance to the app and dashboards.

6. Tech Stack & $0 Cost Breakdown

Layer	Technology	Why	Cost
Services	ASP.NET Core (.NET 10 LTS) Web API, Minimal APIs	High throughput, mature ecosystem	$0
API Gateway	YARP reverse proxy + ASP.NET rate limiting	Routing, JWT validation, token-bucket limits	$0
Inter-service calls	gRPC (internal), REST (external)	Low-latency typed contracts	$0
Ledger database	PostgreSQL 17 + EF Core	ACID, row-level security, check constraints	$0 (Docker)
Read models	Dapper + PostgreSQL read replica / materialised views	Sub-10 ms queries	$0
Cache & locks	Redis 7 (Redlock, SETNX with TTL)	Distributed locks, idempotency cache, OTP store	$0 (Docker)
Event streaming	Apache Kafka (KRaft mode)	Durable event log, replay, consumer groups	$0 (Docker)
Saga orchestration	MassTransit state machines (or custom orchestrator)	Persisted Saga state, timeouts	$0
Resilience	Polly v8	Retry, circuit breaker, timeout, bulkhead	$0
Real-time push	SignalR	Live dashboards and wallet notifications	$0
Frontend	Next.js + TypeScript + Tailwind + shadcn/ui + Recharts	Three web apps	$0

Layer	Technology	Why	Cost
Load generation	k6 + custom .NET simulator workers	Virtual users and traffic presets	$0
Observability	OpenTelemetry, Prometheus, Grafana, Jaeger, Seq	Metrics, distributed tracing, logs	$0 (Docker)
Testing	xUnit, Testcontainers, FsCheck, Pact	Unit, integration, property-based, contract tests	$0
CI/CD	GitHub Actions + docker-compose	Build, test, image publish	$0 (free tier)

7. Backend Services: Detailed Scope

The backend is split into 10 services, each with its own database schema and a single responsibility.

7.1 Identity & KYC Service

- Phone-number + MPIN sign-up, OTP verification (simulated SMS), JWT access + refresh tokens
- Device binding: a new device triggers step-up OTP and a 24-hour transfer limit
- Tiered KYC (Level 0 / 1 / 2) with configurable daily and monthly limits; CNIC verification against a mock NADRA service

7.2 Wallet & Ledger Service (the core)

- Double-entry ledger: every movement writes balanced debit and credit entries in one database transaction
- System accounts for fees, commissions, settlement float, suspense and tax (withholding tax as configurable rule)
- Holds (authorisations) for pending operations: funds are reserved, then captured or released
- Optimistic concurrency (row version) + Redis lock per wallet + a `balance >= 0` check constraint as the last line of defence
- Append-only: entries are never updated or deleted; corrections are reversing entries

7.3 Payments Service (P2P & Raast)

- Wallet-to-wallet transfer (instant, internal ledger only)

- Wallet-to-bank transfer via mock Raast with IBAN/alias lookup; orchestrated as a Saga: Hold → Send to Raast → Await confirmation → Capture or Release
- Request money and split-bill features

7.4 Bill Payment Service

- Consumer-number lookup and bill fetch from mock billers (K-Electric, LESCO, SNGPL, PTCL)
- Payment Saga with retry, timeout and compensation; DLQ for manual review
- Due-date reminders and saved billers

7.5 Top-up & Merchant Service

- Mobile top-up and bundles via mock telcos (Jazz, Zong, Telenor, Ufone)
- Dynamic QR generation for merchants; QR payment with merchant fee split
- Daily merchant settlement batch to bank

7.6 Agent Network Service

- Cash-in / cash-out at agents with agent float management and commission accrual

7.7 Fraud & Risk Engine

- Kafka Streams-style consumer evaluating rules in real time: velocity (e.g. 10 transfers in 60 seconds), new device + large transfer, many-to-one mule patterns, unusual hour/geo
- Risk score per transaction; actions: allow, step-up OTP, hold for review, freeze
- Rules are configurable from the compliance UI without redeploying

7.8 Notification Service

- In-app push via SignalR, simulated SMS and email; template-based, idempotent sends

7.9 Reconciliation & Reporting Service

- Nightly job comparing internal ledger with mock bank settlement files; classifies mismatches
- Daily ledger invariant check: sum of all debits equals sum of all credits
- Statement generation (PDF) and regulatory-style reports

7.10 Simulation & Chaos Service

- Spawns virtual users with realistic behaviour profiles (student, shopkeeper, salaried, fraudster)

- Controls each mock external service: latency, error rate, timeout rate, full outage
- Runs scenario presets S1 to S8 and publishes a pass/fail report

8. Ledger Data Model

Money is stored as integer paisa (BIGINT), never floating point. A balance is derived from entries and cached in the account row, updated in the same transaction.

Table	Key columns	Purpose
`accounts`	id, owner_id, type (customer/merchant/agent/system), currency, balance_paisa, held_paisa, row_version, status	One row per wallet or system account
`journal_transactions`	id, type, idempotency_key, status, created_at, correlation_id	One business operation (e.g. a bill payment)
`journal_entries`	id, transaction_id, account_id, direction (DR/CR), amount_paisa, balance_after	Immutable double-entry lines
`holds`	id, account_id, amount_paisa, expires_at, status (active/captured/released)	Funds reserved during Sagas
`outbox_events`	id, aggregate_id, event_type, payload (JSONB), created_at, processed_at	Transactional Outbox
`idempotency_keys`	key, request_hash, response, expires_at	Duplicate-request protection
`saga_instances`	id, saga_type, current_state, data (JSONB), timeout_at, version	Persisted Saga state
`audit_log`	id, actor, action, payload, prev_hash, hash	Tamper-evident hash chain

Example: Ali pays a Rs. 2,500 K-Electric bill with a Rs. 10 fee

Account	Debit (Rs.)	Credit (Rs.)
Ali's wallet	2,510	
Biller settlement (K-Electric)		2,500
Fee income		10
Total	**2,510**	**2,510**

Database rules enforce the invariants: a trigger rejects any transaction whose entries do not sum to zero, a check constraint blocks `balance_paisa - held_paisa < 0` on customer accounts, and `UPDATE` / `DELETE` on `journal_entries` is revoked from the application role.

9. Functional Requirements

FR-1 Accounts & Identity

- FR-1.1 The system shall register users by phone number with OTP and a 5-digit MPIN.
- FR-1.2 The system shall enforce KYC tier limits on every debit (per transaction, daily, monthly).
- FR-1.3 The system shall bind sessions to a device and require step-up OTP on a new device.

FR-2 Ledger

- FR-2.1 The system shall record every money movement as balanced double-entry lines.
- FR-2.2 The system shall reject any transaction where total debits do not equal total credits.
- FR-2.3 The system shall never allow a customer's available balance to go below zero.
- FR-2.4 The system shall correct errors only through reversing entries.

FR-3 Payments

- FR-3.1 The system shall support wallet-to-wallet, wallet-to-bank (Raast), bill payment, mobile top-up, QR merchant payment and agent cash-in/out.
- FR-3.2 The system shall require an Idempotency-Key on every money-moving request and return the cached response for duplicates within 24 hours.

- FR-3.3 The system shall show each transaction's live status (Pending, Processing, Completed, Failed, Refunded).

FR-4 Sagas & Resilience

- FR-4.1 Multi-step external operations shall run as orchestrated Sagas with persisted state.
- FR-4.2 On external failure the system shall retry up to 3 times with exponential backoff and jitter, then compensate and move the message to a DLQ.
- FR-4.3 Operators shall be able to inspect and replay DLQ items from the back-office.

FR-5 Fraud & Compliance

- FR-5.1 The system shall score every transaction in real time and apply allow / OTP / hold / freeze actions.
- FR-5.2 Compliance officers shall be able to freeze/unfreeze accounts and edit fraud rules without a redeploy.
- FR-5.3 Every privileged action shall be written to a hash-chained audit log.

FR-6 Reconciliation & Reporting

- FR-6.1 The system shall reconcile the ledger against bank settlement files daily and flag mismatches.
- FR-6.2 Users shall be able to download monthly statements as PDF.

FR-7 Simulation

- FR-7.1 The simulator shall spawn up to 10,000 virtual users with configurable behaviour profiles.
- FR-7.2 The simulator shall control latency, error rate and availability of every mock external service at runtime.
- FR-7.3 The simulator shall run scenario presets and produce a pass/fail report with ledger invariant results.

10. Non-Functional Requirements

All targets are measured on a single developer laptop (8 cores, 16 GB RAM) running the full docker-compose stack, and verified by k6 reports and Grafana dashboards.

Category	ID	Specification	How it is verified
Performance	NFR-1.1	Balance and history reads: p95 < 10 ms	k6 + Prometheus histogram
Performance	NFR-1.2	P2P transfer end-to-end: p95 < 150 ms	k6 scenario S1
Throughput	NFR-1.3	Sustain 500 TPS for 10 minutes without errors	k6 ramp test
Integrity	NFR-2.1	0 negative balances and 0 unbalanced journals after any run	Ledger invariant job
Integrity	NFR-2.2	0 lost events when Kafka is down for up to 5 minutes	Scenario S7
Resilience	NFR-3.1	Compensating refund within 3 s of confirmed external failure	Scenario S2 trace in Jaeger
Resilience	NFR-3.2	Circuit breaker opens after 50% failures over 20 calls; half-open after 30 s	Scenario S5
Security	NFR-4.1	Reject tampered webhooks (HMAC SHA-256) in < 1 ms	Unit + load test
Security	NFR-4.2	CNIC, IBAN and phone encrypted at rest (AES-256-GCM)	DB inspection test
Fraud	NFR-5.1	Fraud decision added latency < 20 ms; alert within 2 s	Scenario S6
Observability	NFR-6.1	Every request traceable across all services by correlation ID	Jaeger trace check
Portability	NFR-7.1	`docker compose up` brings up all services in < 3 minutes, $0 cost	Fresh-machine test

11. Engineering Problems & Solutions

P1. Double-spend under concurrency

- Failure: 50 withdrawals hit a Rs. 1,000 wallet in the same millisecond.

- Solution: three layers. A Redis lock (SETNX with TTL and fencing token) on the wallet ID serialises requests; EF Core optimistic concurrency rejects stale writes; a PostgreSQL check constraint makes a negative balance physically impossible.

P2. Dual-write inconsistency

- Failure: the ledger commits but the app crashes before publishing to Kafka, so fraud, notifications and dashboards never hear about it.
- Solution: Transactional Outbox. The ledger rows and the outbox event commit in one transaction; a background relay publishes and marks events processed. Consumers are idempotent (inbox table keyed by event ID), giving effectively-once processing.

P3. External provider failure mid-payment

- Failure: the biller accepts the request but times out before replying. Did the bill get paid?
- Solution: Saga with a hold, not a debit. On timeout the Saga first queries the biller's status endpoint before retrying, so it never double-pays. Only a confirmed failure releases the hold. Unknown outcomes go to a "pending verification" state and are resolved by reconciliation.

P4. Duplicate requests

- Failure: a user on a weak network taps "Send" twice; the mobile client also retries.
- Solution: Idempotency-Key stored in Redis and PostgreSQL with the request hash. Same key + same body returns the cached response; same key + different body returns HTTP 422.

P5. Hot accounts

- Failure: the fee-income and biller-settlement system accounts receive a credit on nearly every transaction, so locking them becomes a bottleneck.
- Solution: split hot system accounts into N sub-accounts (sharded buckets) and sum them for reporting; credits to system accounts skip the per-wallet lock.

P6. Slow dashboards on a busy ledger

- Failure: aggregate queries on the write database slow down transfers.
- Solution: CQRS. Kafka consumers build denormalised read models (balances, histories, per-minute metrics); Dapper serves them; SignalR pushes deltas to the UI.

P7. Cascading failure

- Failure: one slow bank exhausts threads and takes the whole platform down.

- Solution: Polly bulkhead per external provider, timeouts on every call, and circuit breakers; transfers to an unhealthy bank are queued while all other operations continue.

12. Frontend Applications

The frontend exists to make the backend visible and testable. Three Next.js apps share one design system and connect through the API Gateway and SignalR.

12.1 Customer Wallet App (mobile-first web / PWA)

- Onboarding: phone + OTP, MPIN setup, CNIC upload for KYC Level 1
- Home: balance, quick actions (Send, Pay Bills, Top-up, Scan QR, Bank Transfer), recent activity
- Send money to wallet or bank (IBAN / Raast alias) with a confirmation screen and MPIN
- Bill payment: choose biller, enter consumer number, fetch bill, pay
- Live transaction tracker showing each Saga step (Hold placed → Sent to biller → Confirmed / Refunded)
- Notifications, monthly statement download, limits and KYC tier view
- Merchant mode: show QR, today's sales, settlement history

12.2 Operations & Compliance Back-office

- Live operations dashboard: TPS, success rate, p95 latency per service, active Sagas by state
- Saga explorer: search any transaction and see its full timeline and distributed trace link
- DLQ console: inspect, edit-and-replay or manually refund failed messages
- Fraud alerts queue with risk scores; freeze/unfreeze; rule editor
- Customer 360 view: KYC, devices, limits, ledger history
- Reconciliation report: matched, missing, duplicate, amount-mismatch
- Audit log viewer with hash-chain verification badge
- Role-based access: Ops, Compliance, Finance, Admin

12.3 Simulation Control Center (the demo showpiece)

- Virtual user panel: slider for 0 to 10,000 users and a mix of behaviour profiles
- Traffic controls: target TPS, ramp-up curve, transaction mix (P2P / bills / top-up / QR)

- Chaos panel: per external service, sliders for latency and error rate plus a "Kill" switch; buttons to stop Kafka or a database replica
- Scenario launcher: one-click S1 to S8 with a live progress bar
- Live money-flow map of Pakistan: animated transfers between cities, colour-coded by status
- Invariant monitor: always-visible tiles for "Total debits = Total credits", "Negative balances: 0", "Lost events: 0"
- Run report: pass/fail per check, latency charts and a downloadable PDF

13. Security & Compliance

Security is designed in from the first sprint and follows the OWASP API Security Top 10. Regulatory rules are modelled on State Bank of Pakistan branchless-banking practice but are simulated and configurable, not a claim of legal compliance.

- **Authentication:** OTP + MPIN (hashed with Argon2id), short-lived JWTs, rotating refresh tokens, device binding
- **Authorisation:** role-based access in the back-office; PostgreSQL row-level security so a service can only read its tenant's rows
- **Data protection:** AES-256-GCM field encryption for CNIC, IBAN and phone, with keys outside the database; TLS between services
- **Integrity:** HMAC SHA-256 on all incoming webhooks with timestamp and replay window; hash-chained audit log
- **Abuse prevention:** token-bucket rate limiting per user, device and IP; OTP attempt lockout
- **AML/KYC:** tiered limits, velocity rules, freeze workflow and a suspicious-activity report export
- **Secrets:** environment-based secrets for local use, with a documented path to a vault

14. Testing, Observability & DevOps

Practice	Tooling	What it proves
Unit tests	xUnit, FluentAssertions	Domain rules: limits, fees, journal balancing

Practice	Tooling	What it proves
Property-based tests	FsCheck	For any random sequence of transfers, total money in the system is unchanged
Integration tests	Testcontainers (real PostgreSQL, Redis, Kafka)	Outbox delivery, lock behaviour, Saga compensation
Contract tests	Pact	Services and mock providers agree on APIs
Concurrency tests	Parallel xUnit + k6	No double-spend under 50+ parallel debits
Load tests	k6	NFR latency and throughput targets
Chaos tests	Simulation Service presets S1 to S8	Recovery from service, broker and DB failures
Tracing	OpenTelemetry → Jaeger	One trace per payment across all services
Metrics	Prometheus → Grafana	TPS, error rate, Saga states, lock wait time
Logs	Serilog → Seq	Structured logs with correlation ID
CI	GitHub Actions	Build, test, coverage gate (80% on domain), Docker images

Code is organised by Clean Architecture per service (Domain, Application, Infrastructure, API), with architecture tests (NetArchTest) preventing layer violations.

15. Milestones & Timeline

A 16-week plan, built so a working end-to-end demo exists from week 6 and every later phase adds depth.

Phase	Weeks	Deliverable	Exit check
1. Foundations	1–2	Repo, docker-compose, gateway, Identity service, CI pipeline	Sign up and log in end-to-end

Phase	Weeks	Deliverable	Exit check
2. Ledger core	3–4	Double-entry ledger, holds, idempotency, P2P transfer	Concurrency test passes with 0 negative balances
3. Events & CQRS	5–6	Outbox relay, Kafka, read models, SignalR, basic wallet UI	Balance updates live in the UI
4. External payments	7–9	Mock Raast/banks/billers/telcos, Sagas, Polly, DLQ	Scenarios S2 and S5 pass
5. Fraud & compliance	10–11	Fraud engine, rule editor, back-office, audit chain	Scenario S6 passes
6. Simulation Center	12–13	Virtual users, chaos panel, scenario launcher, money-flow map	S1, S3, S4, S7 pass
7. Reconciliation & reports	14	Reconciliation job, statements, run reports	Scenario S8 passes
8. Hardening & demo	15–16	Load tuning, security review, documentation, demo video	All NFR targets met

16. Deliverables, Demo Script & Future Scope

Deliverables

- Source code for 10 backend services and 3 frontend apps in one monorepo
- One-command local deployment (`docker compose up`)
- Architecture document, API reference (OpenAPI/Swagger), ER diagram and ADRs (architecture decision records)
- Test suite with coverage and k6 load-test reports
- Grafana dashboards and a recorded 10-minute demo video

10-minute live demo script

1. Open the wallet app; Ali sends Rs. 500 to a friend. The balance updates live on both screens.
2. Pay a K-Electric bill and show the Saga tracker step by step.

3. In the Control Center, start S1 with 5,000 users at 500 TPS; the money-flow map lights up.
4. Kill the mock HBL service mid-run. The circuit breaker opens and transfers queue; restart it and they drain.
5. Set the K-Electric simulator to 100% timeout. Users get automatic refunds; show the trace in Jaeger.
6. Stop Kafka for 60 seconds. Show outbox rows piling up, then flushing on restart.
7. Launch the SIM-swap fraud ring. Alerts appear on the compliance dashboard and accounts freeze.
8. End the run. The invariant tiles show debits = credits, 0 negative balances, 0 lost events.

Future scope

- Native mobile app (React Native or .NET MAUI)
- ML-based fraud scoring trained on simulator data
- Kubernetes deployment with horizontal autoscaling
- Multi-currency remittances (inbound home remittances)
- Micro-loans and savings pockets on top of the same ledger